

MarketMishmash — Analyse d'architecture & points d'intégration (Phase 1)

Repo cloné dans `/home/ubuntu/github_repos/marketmishmash` (branche `main`).

Objectif : cartographier le projet avant d'intégrer les 3 modules d'analyse de la Phase 1

(`pandas-ta-classic` , `finvizfinance` , `backtesting.py`).

1. Vue d'ensemble du stack

Couche	Technologie	Détails
Frontend	React 18.3 + Vite 7.3	SPA, <code>type: module</code> , JSX en JavaScript (pas de TypeScript)
Style	Tailwind CSS 3.4	Thème sombre custom (couleurs <code>base/panel/accent/up/down</code>)
Graphiques	Recharts 3.10	<code>AreaChart</code> responsive (<code>PriceChart.jsx</code>)
Backend	AUCUN	100 % frontend. Pas de serveur, pas de Python, pas de base de données
Routing	AUCUN	Application mono-page. Pas de <code>react-router</code> . Tout est rendu dans <code>Dashboard.jsx</code>
Persistence	<code>localStorage</code> (navigateur)	Actifs suivis (<code>marketmish-mash_assets</code>) + clés API (<code>marketmishmash_apis</code>)

Point clé : c'est une application **purement client (frontend)**. Il n'existe **ni backend ni page de détail par valeur** aujourd'hui. Les 3 modules Phase 1 sont, eux, des **bibliothèques Python** → un des enjeux majeurs de l'intégration est architectural (voir §5).

2. Sources de données actuelles

Source	Usage	Clé requise	Accès
CoinGecko (<code>services/coingecko.js</code>)	Marché crypto (prix, variation 24h, volume) + historique	Non (clé Demo optionnelle)	Appel direct navigateur (CORS ok)
Finnhub (<code>services/finnhub.js</code>)	Cotation temps réel actions (<code>/quote</code>)	Oui (gratuite)	Appel direct navigateur
Yahoo Finance (<code>services/yahoo.js</code>)	Historique des actions (24h→1an)	Non	Via proxy Vite <code>/yahoo</code> (contourne CORS)

Détails importants :

- L'endpoint payant `stock/candle` de Finnhub n'est **pas** utilisé (déprécié dans le code) → l'historique actions vient de Yahoo.
- Le **proxy Yahoo** (`vite.config.js` , chemin `/yahoo`) n'existe qu'en `npm run dev` / `npm run preview` . Un build statique déployé n'aurait pas ce proxy (nécessiterait un relais). **C'est le précédent qui justifie l'ajout d'un backend** pour la Phase 1.
- Les clés API sont résolues au runtime : **localStorage prioritaire**, repli sur `.env` (`VITE_FINNHUB_API_KEY`). Voir `services/apiStore.js` + hook `useApiConfig.js` .
- `apiRegistry.js` liste déjà des plateformes non branchées (Alpha Vantage, CoinMarketCap, FMP) — utile car **Alpha Vantage / FMP** exposent fondamentaux & indicateurs, à considérer comme alternatives JS.

3. Structure des composants (arborescence réelle)

```

src/
├─ App.jsx                # Rend uniquement <Dashboard/>
├─ main.jsx               # Point d'entrée React
├─ index.css              # Tailwind
├─ components/
│   └─ Dashboard.jsx      # ORCHESTRATEUR : header + section Crypto + section
Actions + modales
│   └─ CryptoCard.jsx     # Carte crypto : prix + variation + volume +
PriceChart + TimeframeSelector
│   └─ StockCard.jsx      # Carte action : prix + variation + PriceChart +
TimeframeSelector
│   └─ PriceChart.jsx     # Graphique Recharts (aire) – réutilisable
│   └─ TimeframeSelector.jsx # Boutons 24h/7j/30j/90j/1an
│   └─ AssetManager.jsx   # Modale d'ajout/suppression d'actifs
│   └─ ApiManager.jsx     # Modale de config des clés API
├─ hooks/
│   └─ useAssets.js        # Liste des actifs (localStorage) + add/remove/reset
│   └─ useApiConfig.js     # Config API (localStorage) + isConfigured/setKey...
│   └─ useCryptoData.js    # Fetch + refresh auto crypto (CoinGecko)
│   └─ useStockData.js     # Fetch + refresh auto actions (Finnhub /quote)
├─ services/
│   └─ coingecko.js finnhub.js yahoo.js # Clients API
│   └─ apiStore.js         # Magasin clés API hors-React (localStorage)
├─ config/
│   └─ assets.json         # Actifs par défaut (1er lancement)
│   └─ apiRegistry.js      # Registre des plateformes connues
│   └─ timeframes.js       # Plages temporelles (mapping CoinGecko days + Yahoo
range/interval)
├─ utils/format.js        # formatPrice / formatPercent / formatVolume /
formatTime

```

Comment une valeur (ticker) est structurée aujourd'hui

- Une crypto = { id (CoinGecko), symbol, name } ; une action = { symbol (Finnhub), name }.
- Chaque valeur est rendue sous forme de **carte** (CryptoCard / StockCard) dans une grille du **Dashboard**. **Il n'y a aucune route ni page de détail** : cliquer sur une carte ne fait rien.
- Le seul état interactif par carte est le **timeframe** du mini-graphique (state local).

4. Ce que demande l'utilisateur (rappel)

- Le **maximum d'analyses utiles** sur les valeurs affichées dans le dashboard.
- Accès via un **onglet déroulant / élément** qui **ouvre une page d'analyse pour la valeur choisie**.
- **Python privilégié** (plus puissant) mais langages de script (JS) acceptés aussi.
- **Pas de fork** en priorité : **s'en inspirer** d'abord et évaluer la difficulté d'intégration.

→ Il faut donc créer un **point d'entrée « Analyser »** par carte, ouvrant une **vue/page de détail** par valeur (nouvelle route ou panneau plein écran), alimentée par les modules Phase 1.

5. Analyse des 3 modules Phase 1 & points d'intégration

Les 3 modules sont des **bibliothèques Python**. L'app étant 100 % frontend JS, deux stratégies :

- **Option A (recommandée) — Ajouter un micro-backend Python** (FastAPI/Flask) qui expose des endpoints d'analyse (`/api/technical` , `/api/fundamentals` , `/api/backtest`). Le frontend appelle ces endpoints depuis la nouvelle page de détail. C'est cohérent avec le précédent du **proxy Vite** (le projet admet déjà une couche serveur en dev/preview) et exploite la pleine puissance Python voulue.
- **Option B — Équivalents JS côté client** (ex. `technicalindicators` en JS) : évite un backend mais perd la richesse de `pandas-ta-classic / backtesting.py` et ne couvre pas `finvizfinance` .

Recommandation : **Option A** (backend Python léger) pour l'analyse technique + fondamentaux + backtest, car les 3 libs demandées sont Python et l'utilisateur veut « le plus d'analyses utiles ».

5.1 pandas-ta-classic — Analyse technique

- **Rôle** : 130+ indicateurs (RSI, MACD, Bollinger, SMA/EMA, ADX, stochastique...) sur un DataFrame OHLCV.
- **Entrée nécessaire** : séries OHLCV. **Disponibles** :
- Actions → déjà récupérables via Yahoo (`fetchStockChartYahoo`) ; côté Python, `yfinance` fournit directement l'OHLCV complet.
- Crypto → CoinGecko `market_chart` (prix) ; pour du vrai OHLCV, endpoint `/ohlcv` de CoinGecko.
- **Point d'intégration** :
- Backend : endpoint `GET /api/technical?symbol=AAPL&timeframe=90d` → renvoie indicateurs calculés.
- Frontend : nouveau service `services/analysis.js` + affichage dans la page de détail (réutiliser `PriceChart` et ajouter des sous-graphes RSI/MACD).
- **Difficulté** : **faible/moyenne**. Le module est mûr ; le principal travail est le backend + le branchement OHLCV.

5.2 finvizfinance — Fondamentaux + news

- **Rôle** : scrape Finviz → fondamentaux (P/E, EPS, market cap, dividende...), news, notes d'analystes, screener. **Actions US uniquement** (pas de crypto).
- **Point d'intégration** :
- Backend : `GET /api/fundamentals?symbol=AAPL` → dict fondamentaux + `GET /api/news?symbol=AAPL` .
- Frontend : onglets « Fondamentaux » et « Actualités » dans la page de détail d'une **action**. Masquer/désactiver ces onglets pour les cryptos.
- **Difficulté** : **faible**. API Python simple (`finvizfinance(ticker)`). Attention au **scraping** (fragilité si Finviz change, throttling) → prévoir cache côté backend.

5.3 backtesting.py — Backtest de stratégies

- **Rôle** : moteur de backtest event-driven sur données OHLC ; stats (rendement, drawdown, Sharpe, win rate) + graphiques interactifs (bokeh).
- **Point d'intégration** :

- Backend : `POST /api/backtest` avec `{symbol, timeframe, strategy, params}` → renvoie métriques JSON (et éventuellement un HTML/plot). Fournir quelques stratégies prêtes (croisement de moyennes, RSI).
- Frontend : onglet « Backtest » dans la page de détail, avec sélecteur de stratégie + tableau de résultats. Rendre les métriques nativement (recharts) plutôt que le plot bokeh pour rester cohérent.
- **Difficulté : moyenne.** Le moteur est simple à lancer, mais l'UX (choix de stratégie/paramètres) et la restitution des résultats demandent du travail frontend.

6. Changements structurels à prévoir pour la page d'analyse

1. **Routing** : ajouter `react-router-dom` (aujourd'hui absent) OU un état `selectedAsset` dans le `Dashboard` ouvrant un **panneau/vue plein écran** (`AssetAnalysis.jsx`). Le routing est plus propre pour « ouvrir une page d'analyse pour la valeur choisie ».
2. **Déclencheur UI** : rendre `CryptoCard / StockCard` cliquables (ou ajouter un bouton «  Analyser ») → navigue vers `/analyse/:type/:symbol`.
3. **Nouveau composant** `AssetAnalysis.jsx` avec **onglets** : Vue d'ensemble · Technique · Fondamentaux (actions) · Backtest.
4. **Nouvelle couche service** `services/analysis.js` pointant vers le backend Python.
5. **Backend Python** (nouveau dossier `backend/`, FastAPI) + proxy Vite `/api` (comme le `/yahoo` existant) pour dev/preview.
6. **Dépendances** à ajouter : frontend `react-router-dom` ; backend `fastapi uvicorn pandas pandas-ta-classic finvizfinance backtesting yfinance`.
7. **Déploiement** : documenter que la page d'analyse nécessite le backend lancé (comme le proxy Yahoo aujourd'hui) — cohérent avec le mode `npm run dev / preview`.

7. Résumé exécutif

- **Stack** : React + Vite + Tailwind + Recharts, **frontend pur**, mono-page, **sans backend ni routing**.
- **Données** : CoinGecko (crypto), Finnhub (cotation actions), Yahoo (historique actions via proxy Vite). Clés API en localStorage (repli `.env`).
- **Aucune page de détail par valeur n'existe** : à créer (route + composant à onglets).
- Les **3 modules Phase 1 sont Python** → recommandation d'un **micro-backend FastAPI** exposant technique / fondamentaux / backtest, appelé par une nouvelle page d'analyse. Cohérent avec le pattern

proxy déjà présent, et conforme au souhait « python + le plus d'analyses utiles », **sans fork** (les libs restent des dépendances).

- **Difficulté d'intégration** : `pandas-ta-classic` faible/moyenne, `finvizfinance` faible (attention scraping/cache), `backtesting.py` moyenne (UX stratégies). L'essentiel de l'effort porte sur la **création du backend** et de la **page d'analyse à onglets** côté frontend.

...